Logistic regression

In contrast to the previous chapter, we now turn towards solving *classification problems with a global approach*. The classic model in this category is **logistic regression**, and we study it in its simplest formulation: when we have a binary (2-class) outcome variable to be predicted.

> *Preliminary comment:* You might be asking yourselves, since we are carrying out a *classification task*, is 'logistic regression' actually a *regression*? The answer is yes, and you can understand this based on our discussion at the end of Section 3.2, Chapter 2 regarding the argmax discretisation. In short, **logistic regression predicts the *probability* that our output belongs to each of the two classes** and then selects the class based on a threshold. In the simplest case, the threshold leads to selecting the outcome with the higher probability, a.k.a. argmax discretisation.

1. Logistic regression

Once again, we set up our problem in the by now standard setting:

$$\mathbf{x}^{(i)} \in \mathbb{R}^p; \quad y^{(i)} \in \{0, 1\} \qquad i = 1, \ldots, N$$

where we have continuous input variables and binary outcome variables to be predicted.

The logistic regression model is global and linear, but adapted to the discrete nature of the outcomes. Since our output variables can only have two outcomes $\{0, 1\}$, one way to understand logistic regression is by considering the so-called *log odds* (or *logit function*):

$$\log \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})} = \log \frac{P(y = 1|\mathbf{x})}{1 - P(y = 1|\mathbf{x})} \tag{3.1}$$

where $P(y = 1|\mathbf{x})$ (respectively $P(y = 0|\mathbf{x})$) is the probability that our variable will take the value 1 (respectively 0).

> **Key message:** The logistic regression model assumes that the logit function (3.1) is a *linear* combination of the input variables:
>
> $$\mathbf{x}^T \boldsymbol{\beta} = \log \frac{P(y = 1|\mathbf{x})}{1 - P(y = 1|\mathbf{x})} \tag{3.2}$$
>
> we have already switched to the shorthand notation where $\mathbf{x}^T$ is a $(p+1)$ length vector $(1, x_1, \ldots, x_p)$, allowing for a bias/intercept term, and $\boldsymbol{\beta} = \begin{pmatrix} \beta_0 & \beta_1 & \cdots & \beta_p \end{pmatrix}^T$.

In other words, logistic regression aims at predicting a ***decision boundary, i.e., a function separating the data predicted in different classes, that is linear*** (see Figure 3.1). Indeed, the decision boundary is given by the set of points where $P(y = 1|\mathbf{x}) = P(y = 0|\mathbf{x})$, hence they form the hyperplane, parametrised by $\boldsymbol{\beta}$, given by $\mathbf{x}^T\boldsymbol{\beta} = 0$.

With some rearranging of (3.2), we also see:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{x}^T\boldsymbol{\beta}}} = h\left(\mathbf{x}^T\boldsymbol{\beta}\right) =: h_{\boldsymbol{\beta}}(\mathbf{x}) \tag{3.3}$$

This is called the *logistic function*, which is one of the key functions in statistical learning—it will reappear down the line when you look at neural networks. Clearly, we also have:

$$P(y = 0|\mathbf{x}) = 1 - h_{\boldsymbol{\beta}}(\mathbf{x}) \tag{3.4}$$

You already know how the logistic function looks but for a reminder, see Figure 3.1.

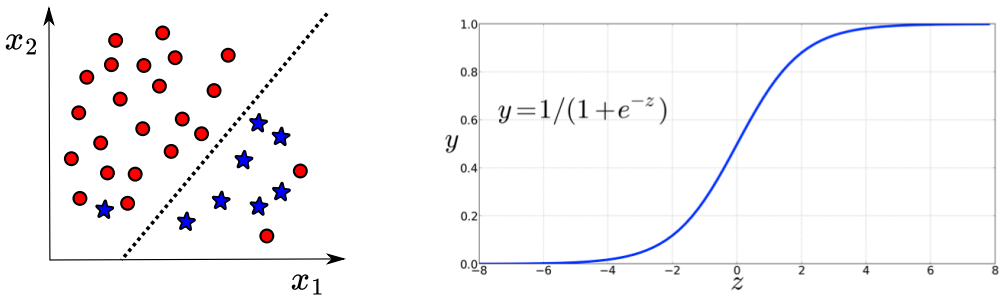


Figure 3.1. (Left) Logistic regression learns a linear decision boundary (a line in the two-dimensional case). (Right) The logistic function for a one-dimensional variable z.

The above expressions (3.3)–(3.4) can be immediately understood as the result of assuming that our output variable is a *Bernoulli* random variable with a probability of success $P(y = 1|\mathbf{x})$ (and of failure $P(y = 0|\mathbf{x})$) :

$$P(Y = y \mid \mathbf{x}, \boldsymbol{\beta}) = \big(h_{\boldsymbol{\beta}}(\mathbf{x})\big)^y \big(1 - h_{\boldsymbol{\beta}}(\mathbf{x})\big)^{1-y}$$

In summary, the logistic regression model is a probabilistic model that makes the specific assumption that **the probability distribution of the output conditional on the data features is a Bernoulli distribution** with probability of success that depends on the parameters $\boldsymbol{\beta}$ and is given by the logistic function $h_{\boldsymbol{\beta}}(\mathbf{x})$ (3.3).

> *Quick Aside:* There are many ways to introduce and motivate logistic regression within broader families of approaches to classification: indeed, logistic regression is one member of the family of linear classification methods (i.e., methods aimed at learning linear decision boundaries), and of the one of probabilistic classifiers that rely on specific assumptions on the conditional probability density of output labels. In Chapter 4 of Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, you can read more about logistic regression (section 4.4) in comparison to other methods from these families. You will also find more information on extensions of logistic regression to multi-class classification.

1.1. Estimating the parameters. To estimate the parameters $\boldsymbol{\beta}$ **we maximise the log-likelihood of the model over the data** (where here the log-likelihood is the one

of outputs conditional on the data features). Given that the N samples can be assumed to be independent, we can factorise probabilities to get:

$$P\big(\{y^{(i)}\} \mid \{x^{(i)}\}, \boldsymbol{\beta}\big) = \prod_{i=1}^{N} \big(h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\big)^{y^{(i)}} \big(1 - h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\big)^{1-y^{(i)}}$$

This leads us to the log-likelihood:

$$\mathcal{L} = \sum_{i=1}^{N} y^{(i)} \log h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log \big(1 - h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\big) \tag{3.5}$$

The optimisation follows a similar pattern to what we saw in linear regression in Chapter 1. As with linear regression, we set $\nabla_{\boldsymbol{\beta}}\mathcal{L} = 0$ to find the maximum. Note that:

$$P(y = 0|\mathbf{x}) = 1 - h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)}) = \frac{e^{-(\mathbf{x}^{(i)})^T\boldsymbol{\beta}}}{1 + e^{-(\mathbf{x}^{(i)})^T\boldsymbol{\beta}}} = e^{-(\mathbf{x}^{(i)})^T\boldsymbol{\beta}}\ h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})$$

so that:

$$\log\Big(1 - h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\Big) = -(\mathbf{x}^{(i)})^T\boldsymbol{\beta} + \log h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})$$

We can now use these expressions (fill in the easy steps) to simplify the log-likelihood to get:

$$\mathcal{L} = \sum_{i=1}^{N} \log h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)}) - (1 - y^{(i)})(\mathbf{x}^{(i)})^T\boldsymbol{\beta}$$

In order to obtain the gradient, first note that:

$$\begin{aligned}\nabla_{\boldsymbol{\beta}}\big(\log h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\big) &= \frac{e^{-(\mathbf{x}^{(i)})^T\boldsymbol{\beta}}}{1 + e^{-(\mathbf{x}^{(i)})^T\boldsymbol{\beta}}}\mathbf{x}^{(i)} \\ &= \big(1 - h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\big)\mathbf{x}^{(i)}\end{aligned}$$

Finally, using the above, we can write down the gradient of the log-likelihood:

$$\nabla_{\boldsymbol{\beta}}\mathcal{L} = \sum_{i=1}^{N} \big(1 - h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\big)\mathbf{x}^{(i)} - (1 - y^{(i)})\mathbf{x}^{(i)} = \sum_{i=1}^{N} \big(y^{(i)} - h_{\boldsymbol{\beta}}(\mathbf{x}^{(i)})\big)\mathbf{x}^{(i)}$$

Now, we look for the maximum (i.e., the equivalent of the Normal Equation in linear regression), which is where:

$$\nabla_{\boldsymbol{\beta}}\mathcal{L}|_{\boldsymbol{\beta}^\star_{\log}} = \sum_{i=1}^{N} \Big(y^{(i)} - h_{\boldsymbol{\beta}^\star_{\log}}(\mathbf{x}^{(i)})\Big)\,\mathbf{x}^{(i)} = \mathbf{0} \tag{3.6}$$

where the notation $\boldsymbol{\beta}^\star_{\log}$ indicates that this is the solution of the minimisation for logistic regression. Unlike the case of linear regression, this system of $p + 1$ equations can not be solved analytically. However, **since $\mathcal{L}$ is concave** (try to show it as an exercise), **standard optimisation algorithms (e.g., gradient ascent) are able to find the maximum**.

In practice, one can formulate the logistic regression learning task as the minimisation of a mean sample loss given by the mean negative log-likelihood:

$$E(L) = -\frac{1}{N}\sum_{i=1}^{N} y^{(i)} \log h_{\boldsymbol{\beta}}(\boldsymbol{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\boldsymbol{\beta}}(\boldsymbol{x}^{(i)}))$$

which gives the same solution as maximising the log-likelihood (3.5).

Once we have obtained the $\boldsymbol{\beta}^\star_{\log}$ that fulfils the equation (3.6) by gradient methods, we can then plug this into the equation (3.3) to get the 'probability of success'.

Key message: In summary, training a logistic regression model gives a probabilistic estimator for the input variables, hence a *real* outcome (a probability):

$$P(y=1|\mathbf{x}^{\text{in}}) = h_{\boldsymbol{\beta}^\star_{\log}}(\mathbf{x}^{\text{in}}) = h\big((\mathbf{x}^{\text{in}})^T\boldsymbol{\beta}^\star_{\log}\big) \in [0,1] \tag{3.7}$$

To get a hard classifier, we perform a *discretisation step* by defining a threshold τ for the classifier such that the sample is assigned to each of the two classes:

$$P(y=1 \mid \mathbf{x}^{\text{in}}) > \tau \implies \mathbf{x}^{\text{in}} \in \{1\}, \tag{3.8}$$

The threshold is a hyperparameter of the model, which can be tuned *a posteriori* to improve the performance of the method, as we will see below. Clearly, if we take $\tau = \frac{1}{2}$, then the input variable is assigned to the class with maximum probability of the two classes (hence equivalent to argmax).

Quick aside: We can also summarise the solution to the logistic regression problem in matrix-vector form, as follows. Consider the vector version of (3.3) where $\mathbf{h}(\mathbf{X}\boldsymbol{\beta}^\star_{\log})$ is a N-dimensional vector function where each individual entry is:

$$h_i(\mathbf{X}\boldsymbol{\beta}^\star_{\log}) = \left(\frac{1}{1+e^{-(\mathbf{x}^{(i)})^T\boldsymbol{\beta}^\star_{\log}}}\right), \quad i=1,\dots,N$$

For some data set $\mathbf{X}$ (dimension $N \times (p+1)$), we have obtained the following probabilistic estimator:

$$\mathbf{h}(\mathbf{X}\boldsymbol{\beta}^\star_{\log}) = \hat{\mathbf{y}}$$

where $\hat{\mathbf{y}}$ is a vector of dimension $N \times 1$. The 'Normal Equation'-equivalent (3.6) is:

$$\mathbf{X}^T\big[\mathbf{y} - \mathbf{h}(\mathbf{X}\boldsymbol{\beta}^\star_{\log})\big] = 0$$

Compare this to the Normal Equation for linear regression, which was:

$$\mathbf{X}^T\big[\mathbf{y} - \mathbf{X}\boldsymbol{\beta}^\star_{\text{LS}}\big] = 0.$$

2. Quality function for the classifier: Confusion matrix and Receiver Operating Characteristic (ROC)

We mentioned in Chapter 2, when discussing kNN classifiers, that a quality function (or error function) needs to be developed to quantify the performance of the model, since error functions like the MSE are not appropriate for discrete outcomes.

2.1. Confusion matrix and associated quality measures. We now discuss how to obtain quality measures for binary classification, as needed for logistic regression. To do this, we introduce the *confusion matrix*.

For the logistic regression with outcomes in $\{0,1\}$, Table 3.1 shows an overview of the different fields of a confusion matrix. With the four numbers (TP, FP, FN, TN) of this confusion matrix, one can create different 'quality measures' for the classifier. Below we show several such measures, all of which are used for different applications (i.e., depending on what is the objective of our computations and the type of data and application):

The *true positive rate* or *sensitivity* or *recall*:

$$\text{TPR} = \frac{\text{TP}}{\text{P}} = \frac{\text{TP}}{\text{TP}+\text{FN}} = \text{recall}$$

		Truth	
		1	0
Predicted	1	true positives (TP)	false positives (FP)
	0	false negatives (FN)	true negatives (TN)

Table 3.1. Confusion matrix schematic. A good model will have most cases on the diagonal, i.e. either TP or TN. Note that FP is sometimes called the *Type I error* and FN is sometimes the *Type II error*. The sum over the first column gives the total amount of actual positives in the data P = TP+FN; the sum over the second column gives the total amount of actual negatives N = FP+TN.

The *true negative rate* or *specificity*:

$$\mathrm{TNR} = \frac{\mathrm{TN}}{\mathrm{N}} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}$$

The *accuracy*:

$$\text{accuracy} = \frac{\mathrm{TP} + \mathrm{TN}}{N_{\text{validation}}}$$

where $N_{\text{validation}}$ is the number of data on which the model has been 'validated, and is simply the sum of all entries of the confusion matrix ($N_{\text{validation}} = \mathrm{FN} + \mathrm{FP} + \mathrm{TN} + \mathrm{TP}$). Furthermore, the *precision* (or *Positive Predictive Value*):

$$\text{precision} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

And finally, the *F-score*:

$$\mathrm{F} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

F-score is the harmonic mean of precision and recall. It rewards models with similar recall and precision. Which of these functions is chosen depends on the particular problem—how much tolerance of FPs or FNs our application can accept. This will depend heavily on the field of application.

The case of imbalanced data. In imbalanced datasets, the sizes of the different data subsets are expected to be different. In the case of binary classification, one typically has a minority class (positives) which are present in much lower quantity than the majority class (negatives). This setting represents what is called a 'rare-class learning problem', which can appear in many applications, for example: the medical diagnoses of a disease (where disease cases are rarer than healthy cases); problems of anomaly detection (detection of frauds among normal financial transactions, detection of risk situations, errors in texts, etc.).

If one cares about the retrieval of the positive class, and less about the discrimination or balance of retrieval between classes, **the recommendation is to focus on measures that do not depend on the number of True Negatives (TN)**, being they many more than the positives in the cases under consideration. Such measures are recall, precision and F-score. Precision is particularly important since it gives the proportion of predicted positives that are true positives, hence it takes into account false positives as a source of misclassification. Precision tells how much we can trust the model when it predicts positives - which is what we are interested in in rare-class learning problems.

Accuracy and AUC on the other hand can assume misleadingly high performance: for instance, the accuracy can be high due to the fact that the majority class only is well

predicted. **Generally speaking, recall, precision and F-score are the measures more suitable for assessing the prediction of a minority class.**

In addition, there exist *ad hoc* adjustments of the classification measures to better handle the imbalanced data case, an example is 'balanced accuracy', which is a re-definition of accuracy that gives more weight to the minority class. It is given by:

$$\text{Balanced Accuracy} = \frac{\frac{\text{TP}}{\text{P}} + \frac{\text{TN}}{\text{N}}}{2}$$

i.e., the true positives (respectively, true negatives) are re-weighted by 1/P (respectively, 1/N), where P (respectively, N) is the total number of actual positives (respectively, negatives) in the dataset (see Table 3.1).

Importantly, imbalanced data are a vast area of research that involves also strategies at the level of algorithms and of the use of the data during the training that should be applied to ensure a good performance in this imbalanced cases (e.g., one strategy is to re-balance class distribution by weighting data, or by re-sampling them either by under-sampling the majority of the class, or over-sampling the minority class, or a mix of both.) For a discussion of these more advanced aspects see Sun et al., *Classification of imbalanced data: a review*, International Journal of Pattern Recognition and Artificial Intelligence (2009), which is available as additional reading.

> *Quick Aside.* In the case of **multi-class classification**, one can build the equivalent of the confusion matrix (here called contingency matrix), with the correct predictions along the diagonal and the incorrect predictions in the off-diagonal. There are some measures that are easily generalisable such as the accuracy (given by the number of correct predictions normalised by all predictions); for other measures, one can always use the same measures defined above for pairwise comparisons between classes and then take the average. The terminology *micro* or *macro* average is typically used depending on whether the averaging takes into account the class relative proportion or not - for a detailed discussion, see e.g. Grandini et al., *Metrics for multi-class classification: an overview* (2020) in 'additional reading'.

2.2. ROC curve. In addition to the 'zoo' of various quality functions that can be used to gauge a model's quality, a widely used representation is the *receiver operating characteristic (ROC) curve*, which allows to represent the balance of errors graphically, see Figure 3.2.

Remember that our classifier has a hyperparameter (the threshold τ) that can be tuned to fit our purpose. In Figure 3.2, we scan across a range of values of τ and plot the true positive rate (TPR) against the false positive rate (FPR) to obtain a curve that ideally bulges towards the point where TPR $= 1$ and FPR $= 0$. The area under the curve is then an overall measure of quality for the model. Trivially, one can see that we want the area under the curve to be > 0.5 since that will mean that we are performing better than a random classifier.

The ROC curve can therefore be used also to select the hyperparameter τ by establishing the balance between TPR and FNR appropriate for our problem.

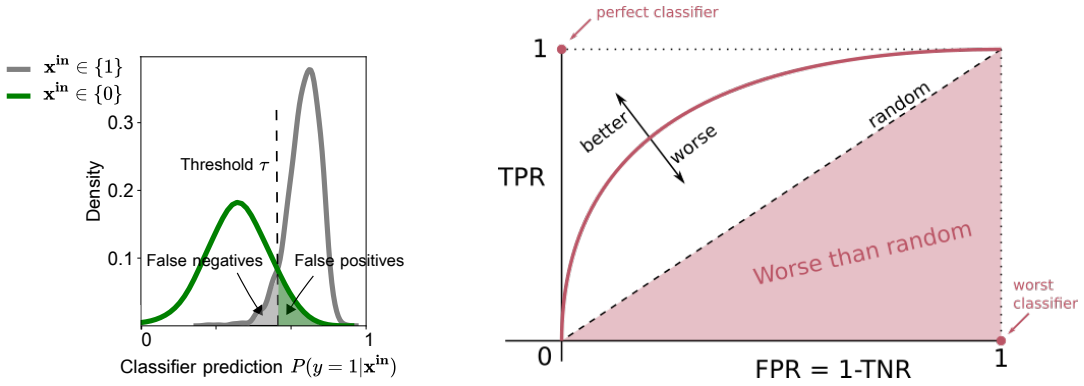


Figure 3.2. (Left) Cartoon illustration of a distribution of probabilistic predictions by a classifier. (Right) The receiver operating characteristic (ROC) curve. When scanning across a range of values for the threshold (τ) parameter of a logistic regression (or some other binary outcome model) and plotting the true positive rate (TPR) against the false positive rate (FPR), we may expect to get a curve similar to the thick upwards curving line in red. If this curve is above the diagonal (dashed) line, the model is doing better than a completely random classifier and vice versa.